

R Programming for Data Analysis

DAY 3 — Data Frames & Tibbles

Day 3 Objective

In **Day 1**, we learned R basics, variables, data types, operators and functions.

In **Day 2**, we learned vectors, indexing, filtering, lists and factors.

Today we move to one of the **most important concepts in R for Data Analysts: Data Frames**.

By the end of Day 3, students should be able to:

- Understand Data Frames
 - Create Data Frames
 - Understand rows and columns
 - Access columns
 - Access rows
 - Filter records
 - Add new columns
 - Modify columns
 - Remove columns
 - Sort data
 - Use `summary()`, `str()` and `head()`
 - Understand Tibbles
 - Perform basic employee/sales analysis
-

1. What is a Data Frame?

A **Data Frame** is a two-dimensional table-like structure in R that stores data in rows and columns.

It is similar to a table in:

- Excel
- SQL
- Power BI
- Database tables

For example:

Employee	Age	Department	Salary
Amit	25	Sales	30000
Rahul	28	IT	45000
Priya	26	HR	38000
Neha	30	IT	52000

This can be stored in R as a **Data Frame**.

2. Why are Data Frames Important?

Real-world data normally looks like a table.

For example:

Customer Data

Customer_ID	Name	City	Sales
C001	Amit	Mumbai	25000
C002	Rahul	Pune	32000
C003	Priya	Delhi	28000

Employee Data

Employee_ID	Name	Department	Salary
E001	Amit	Sales	35000
E002	Rahul	IT	45000

E003	Neha	HR	40000
------	------	----	-------

Sales Data

Order_ID	Product	Quantity	Sales
O001	Laptop	2	120000
O002	Mouse	5	5000
O003	Keyboard	3	7500

Therefore, **Data Frames** are extremely important for **Data Analysis** in **R**.

3. Creating a Data Frame

Use the `data.frame()` function.

```
employees <- data.frame(
  Name = c("Amit", "Rahul", "Priya", "Neha"),
  Age = c(25, 28, 26, 30),
  Department = c("Sales", "IT", "HR", "IT"),
  Salary = c(30000, 45000, 38000, 52000)
)
```

Display the Data Frame:

```
employees
```

Output:

```
   Name Age Department Salary
1  Amit  25      Sales 30000
2 Rahul  28         IT  45000
3 Priya  26         HR  38000
4  Neha  30         IT  52000
```

4. Understanding Rows and Columns

Our Data Frame has:

Rows

Each row represents one **record**.

Example:

```
Amit  
Rahul  
Priya  
Neha
```

Each employee is one record.

Columns

Each column represents one **variable/field**.

```
Name  
Age  
Department  
Salary
```

Therefore:

Row = Record

Column = Variable

This concept is important when working with Excel, SQL, Power BI and R.

5. Checking the Number of Rows

Use:

```
nrow(employees)
```

Output:

```
4
```

There are 4 employees.

6. Checking the Number of Columns

Use:

```
ncol(employees)
```

Output:

```
4
```

There are 4 columns.

7. Checking Both Rows and Columns

Use:

```
dim(employees)
```

Output:

```
4 4
```

Meaning:

```
4 Rows  
4 Columns
```

8. Checking Column Names

Use:

```
names(employees)
```

Output:

```
"Name" "Age" "Department" "Salary"
```

Another function:

```
colnames(employees)
```

9. Viewing the First Rows

Use:

```
head(employees)
```

`head()` displays the first few rows.

You can specify the number:

```
head(employees, 2)
```

This displays the first 2 rows.

10. Viewing the Last Rows

Use:

```
tail(employees)
```

For example:

```
tail(employees, 2)
```

Displays the last 2 rows.

11. Understanding the Structure

Use:

```
str(employees)
```

`str()` means **structure**.

It tells us:

- Number of observations
- Number of variables
- Column names
- Data types

Example:

```
'data.frame': 4 obs. of 4 variables
```

This is extremely useful when you receive a new dataset.

12. Summary of Data

Use:

```
summary(employees)
```

It provides a quick statistical summary of the Data Frame.

For numeric columns, you may see:

- Minimum
- 1st Quartile
- Median
- Mean
- 3rd Quartile
- Maximum

13. Accessing a Column

There are several ways.

Method 1 — `$`

```
employees$Name
```

Output:

```
"Amit" "Rahul" "Priya" "Neha"
```

Salary:

```
employees$Salary
```

Department:

```
employees$Department
```

14. Using `[ ]`

We can also access columns using brackets.

```
employees["Salary"]
```

Notice that this returns a Data Frame.

To get the actual vector:

```
employees[["Salary"]]
```

15. `$` vs `[[ ]]` vs `[ ]`

Syntax	Result
<code>employees\$Salary</code>	Column vector
<code>employees[["Salary"]]</code>	Column vector
<code>employees["Salary"]</code>	Data Frame

For beginners:

```
employees$Salary
```

is the easiest way to access a column.

16. Accessing a Particular Row

Suppose we want the first row:

```
employees[1, ]
```

The comma is important.

```
[row, column]
```

So:

```
employees[1, ]
```

means:

```
Row 1, all columns
```

17. Accessing a Particular Cell

Suppose we want Rahul's salary.

Rahul is row 2 and Salary is column 4.

```
employees[2, 4]
```

Output:

```
45000
```

General syntax:

```
dataframe[row, column]
```

18. Selecting Specific Rows and Columns

First two rows:

```
employees[1:2, ]
```

First two columns:

```
employees[, 1:2]
```

Rows 1 and 3:

```
employees[c(1, 3), ]
```

Columns Name and Salary:

```
employees[, c("Name", "Salary")]
```

19. Filtering Data

Filtering is one of the **most important Data Analyst operations**.

Suppose:

```
employees$Salary > 40000
```

This returns:

```
FALSE TRUE FALSE TRUE
```

Now use it to filter the Data Frame:

```
employees[employees$Salary > 40000, ]
```

Output:

Name	Age	Department	Salary
Rahul	28	IT	45000
Neha	30	IT	52000

20. Filter by Department

Find employees from IT:

```
employees[employees$Department == "IT", ]
```

Find Sales employees:

```
employees[employees$Department == "Sales", ]
```

Find HR employees:

```
employees[employees$Department == "HR", ]
```

21. Multiple Conditions

We can use:

```
& → AND  
| → OR
```

AND

Find IT employees earning more than ₹40,000:

```
employees[  
  employees$Department == "IT" &
```

```
employees$Salary > 40000,  
]
```

Both conditions must be TRUE.

OR

Find employees from IT or Sales:

```
employees[  
  employees$Department == "IT" |  
  employees$Department == "Sales",  
]
```

22. Adding a New Column

Suppose we want to calculate annual salary.

```
employees$Annual_Salary <- employees$Salary * 12
```

Now:

```
employees
```

The Data Frame contains a new column:

Name	Salary	Annual_Salary
Amit	30000	360000
Rahul	45000	540000
Priya	38000	456000
Neha	52000	624000

This is an example of creating a **derived column**.

23. Adding Bonus Column

Suppose employees receive a 10% bonus.

```
employees$Bonus <- employees$Salary * 0.10
```

Now we have:

```
Salary  
Bonus  
Annual Salary
```

24. Modifying a Column

Suppose Amit's salary changes.

```
employees$Salary[1] <- 35000
```

Now Amit's salary becomes:

```
35000
```

25. Renaming a Column

We can change a column name.

```
names(employees)[1] <- "Employee_Name"
```

Now the first column is:

```
Employee_Name
```

26. Removing a Column

Suppose we don't need the Bonus column.

```
employees$Bonus <- NULL
```

The column is removed.

Another method:

```
employees$Bonus <- NULL
```

27. Sorting Data

Suppose we want to sort employees according to salary.

Ascending

```
employees[order(employees$Salary), ]
```

Descending

```
employees[order(employees$Salary, decreasing = TRUE), ]
```

This is useful when finding:

- Highest-paid employee
- Lowest-paid employee
- Top sales
- Bottom sales

28. Finding the Highest Salary Employee

```
employees[  
  which.max(employees$Salary),  
]
```

This returns the row containing the highest salary.

29. Finding the Lowest Salary Employee

```
employees[  
  which.min(employees$Salary),  
]
```

30. Data Frame vs Vector

Vector

```
salary <- c(30000, 45000, 38000, 52000)
```

Stores one type of information.

Data Frame

```
employees <- data.frame(
  Name = c("Amit", "Rahul", "Priya", "Neha"),
  Age = c(25, 28, 26, 30),
  Salary = c(30000, 45000, 38000, 52000)
)
```

Stores multiple variables together.

Think:

Vector = One column

Data Frame = Complete table

31. What is a Tibble?

A **Tibble** is a modern version of a Data Frame commonly used in the tidyverse ecosystem.

Example:

```
library(tibble)

employees <- tibble(
  Name = c("Amit", "Rahul", "Priya"),
  Age = c(25, 28, 26),
  Salary = c(30000, 45000, 38000)
)
```

Display:

```
employees
```

32. Data Frame vs Tibble

Data Frame	Tibble
Base R structure	Modern tidyverse structure

<code>data.frame()</code>	<code>tibble()</code>
Traditional output	Cleaner output
Widely supported	Common in modern R analysis

Students should understand both, but later in the course we will mainly work with **tibbles** + **dplyr**.

33. Complete Practical Example — Employee Dataset

Let's create a more realistic dataset.

```
employees <- data.frame(
  Employee_ID = c("E001", "E002", "E003", "E004", "E005"),
  Name = c("Amit", "Rahul", "Priya", "Neha", "Karan"),
  Age = c(25, 28, 26, 30, 27),
  Department = c("Sales", "IT", "HR", "IT", "Sales"),
  Salary = c(30000, 45000, 38000, 52000, 35000)
)
```

View Dataset

```
employees
```

Average Salary

```
mean(employees$Salary)
```

Highest Salary

```
max(employees$Salary)
```

Lowest Salary

```
min(employees$Salary)
```

Employees earning more than ₹40,000

```
employees[employees$Salary > 40000, ]
```

IT Department Employees

```
employees[employees$Department == "IT", ]
```

Add Annual Salary

```
employees$Annual_Salary <- employees$Salary * 12
```

Sort by Salary

```
employees[order(employees$Salary, decreasing = TRUE), ]
```



DAY 3 PRACTICE QUESTIONS

Practice 1 — Create a Data Frame

Create a Data Frame named `students` with:

```
Name  
Age  
Course  
Marks
```

Use at least 5 students.

Practice 2 — Basic Information

Using `students`, find:

1. Number of rows
2. Number of columns
3. Column names
4. Structure

5. Summary

Use:

```
nrow()  
ncol()  
names()  
str()  
summary()
```

Practice 3 — Indexing

Using the student Data Frame:

1. Display first student.
 2. Display third student.
 3. Display only the `Name` column.
 4. Display only the `Marks` column.
 5. Display first three students.
-

Practice 4 — Filtering

Find students:

1. Scoring more than 70
 2. Scoring less than 50
 3. Scoring exactly 90
 4. Age greater than 20
-

Practice 5 — Employee Data

Create:

```
employees <- data.frame(  
  Name = c("Amit", "Rahul", "Priya", "Neha", "Karan"),  
  Age = c(25, 28, 26, 30, 27),  
  Department = c("Sales", "IT", "HR", "IT", "Sales"),  
  Salary = c(30000, 45000, 38000, 52000, 35000)  
)
```

Perform:

1. Find average salary.

2. Find highest salary.
 3. Find lowest salary.
 4. Display IT employees.
 5. Display employees earning above ₹40,000.
 6. Sort salary from highest to lowest.
 7. Add `Annual_Salary` .
 8. Add `Bonus` of 10%.
 9. Find the employee with the highest salary.
 10. Find the employee with the lowest salary.
-

★ DAY 3 ASSIGNMENT — Sales Data Analysis

Create this Data Frame:

```
sales <- data.frame(  
  Order_ID = c("0001", "0002", "0003", "0004", "0005", "0006"),  
  Product = c("Laptop", "Mouse", "Keyboard", "Laptop", "Monitor",  
"Mouse"),  
  Category = c("Electronics", "Accessories", "Accessories",  
               "Electronics", "Electronics", "Accessories"),  
  Quantity = c(2, 5, 3, 1, 2, 4),  
  Sales = c(120000, 5000, 7500, 60000, 40000, 4000)  
)
```

Analyze the dataset:

- Q1.** Display the complete dataset.
- Q2.** Find the number of orders.
- Q3.** Find total sales.
- Q4.** Find average sales per order.
- Q5.** Find the highest sales order.
- Q6.** Find the lowest sales order.
- Q7.** Display orders where sales are greater than ₹20,000.
- Q8.** Display only Electronics orders.
- Q9.** Sort orders by Sales from highest to lowest.
- Q10.** Create a new column:

```
Revenue_Per_Unit = Sales / Quantity
```

DAY 3 QUICK REVISION

Creating Data Frame

```
data.frame()
```

Rows

```
nrow()
```

Columns

```
ncol()
```

Dimensions

```
dim()
```

Column Names

```
names()
```

First Rows

```
head()
```

Last Rows

```
tail()
```

Structure

```
str()
```

Summary

```
summary()
```

Access Column

```
data$column
```

Access Row

```
data[1, ]
```

Access Cell

```
data[1, 2]
```

Filter

```
data[data$Salary > 40000, ]
```

Add Column

```
data$Annual_Salary <- data$Salary * 12
```

Sort

```
data[order(data$Salary, decreasing = TRUE), ]
```

DAY 3 LEARNING OUTCOME

The key progression so far is:

Day 1: R Fundamentals

↓

Day 2: Vectors & Data Structures

↓

Day 3: Data Frames & Tables

↓

Day 4: Control Statements + Functions

↓

Day 5: Importing & Cleaning Real-World Data

↓

Day 6: `dplyr` Data Manipulation

↓

Day 7: `ggplot2` Visualization

The **most important skill from Day 3** is understanding this structure:

```
data[row, column]
```

because it forms the foundation for working with tabular data in R.

Today 3:23 PM

Day 3

R Programming for Data Analysis

DAY 3 — DATA FRAMES & TIBBLES

Learning Objectives

After completing Day 3, students will be able to:

- Understand what a Data Frame is
- Create a Data Frame
- Understand rows and columns
- Access rows and columns
- Add, modify and remove columns
- Filter records
- Sort data
- Find basic statistics
- Understand Tibbles
- Perform basic business data analysis

1. Introduction to Data Frames

In real-world Data Analysis, data is usually available in **tabular format**, similar to an Excel worksheet or SQL table.

For example:

Employee e_ID	Name	Department	Age	Salary
E001	Amit	Sales	25	30000
E002	Rahul	IT	28	45000

E003	Priya	HR	26	38000
E004	Neha	IT	30	52000
E005	Karan	Sales	27	35000

In R, this type of data can be stored in a **Data Frame**.

Definition

A Data Frame is a two-dimensional data structure in R consisting of rows and columns.

It is one of the most important structures for a Data Analyst working with R.

2. Data Frame vs Excel

A Data Frame is similar to an Excel worksheet.

Excel	R Data Frame
Workbook /Sheet	Data Frame
Column	Variable
Row	Record/Observation
Cell	Individual value
Filter	Filtering condition
Formula	R expression/function

For example:

Excel

```
Salary > 40000
```

R

```
employees[employees$Salary > 40000, ]
```

3. Why Data Frames Are Important

Most data-analysis tasks involve **multiple columns and multiple records**.

For example:

Customer Data

```
Customer_ID  
Customer_Name  
City  
Age  
Purchase
```

Sales Data

```
Order_ID  
Product  
Quantity  
Sales  
Date
```

Employee Data

```
Employee_ID  
Name  
Department  
Salary  
Experience
```

All these datasets can be represented using Data Frames.

4. Creating a Data Frame

We use:

```
data.frame()
```

Example

```
employees <- data.frame(  
  Name = c("Amit", "Rahul", "Priya", "Neha"),  
  Age = c(25, 28, 26, 30),  
  Department = c("Sales", "IT", "HR", "IT"),  
  Salary = c(30000, 45000, 38000, 52000)  
)
```

Display the Data Frame:

```
employees
```

Output:

	Name	Age	Department	Salary
1	Amit	25	Sales	30000
2	Rahul	28	IT	45000
3	Priya	26	HR	38000
4	Neha	30	IT	52000

5. Understanding Rows and Columns

Consider:

```
employees
```

Rows

Each employee represents one **record/observation**.

```
Amit  
Rahul  
Priya  
Neha
```

Columns

Each column represents a **variable**.

```
Name
Age
Department
Salary
```

Therefore:

```
Row = Record / Observation
```

```
Column = Variable / Field
```

6. Data Frame Indexing

The basic syntax is:

```
data[row, column]
```

For example:

```
employees[1, 2]
```

means:

```
Row 1, Column 2
```

Output:

```
25
```

7. Selecting a Row

To select the first row:

```
employees[1, ]
```

The comma means:

```
Give me all columns.
```

Second row:

```
employees[2, ]
```

Third row:

```
employees[3, ]
```

8. Selecting a Column

The easiest method is `$`.

```
employees$Name
```

Output:

```
"Amit" "Rahul" "Priya" "Neha"
```

Salary:

```
employees$Salary
```

Department:

```
employees$Department
```

9. Selecting Multiple Columns

```
employees[, c("Name", "Salary")]
```

This displays only:

```
  Name  
Salary
```

Another method:

```
employees[c("Name", "Salary")]
```

10. Selecting Specific Rows and Columns

First two rows and first two columns:

```
employees[1:2, 1:2]
```

Rows 1 and 3:

```
employees[c(1, 3), ]
```

Columns 1 and 4:

```
employees[, c(1, 4)]
```

11. Number of Rows

Use:

```
nrow(employees)
```

Output:

```
4
```

12. Number of Columns

Use:

```
ncol(employees)
```

Output:

```
4
```

13. Dimensions

Use:

```
dim(employees)
```

Output:

```
4 4
```

Meaning:

```
4 Rows  
4 Columns
```

14. Column Names

Use:

```
names(employees)
```

Output:

```
"Name" "Age" "Department" "Salary"
```

You can also use:

```
colnames(employees)
```

15. First Few Records

Use:

```
head(employees)
```

By default, `head()` displays the first six rows when there are enough rows.

You can specify the number:

```
head(employees, 2)
```

This displays the first two records.

16. Last Few Records

Use:

```
tail(employees)
```

Or:

```
tail(employees, 2)
```

This displays the last two records.

17. Understanding Dataset Structure

Use:

```
str(employees)
```

`str()` means **structure**.

It helps us understand:

- Number of rows

- Number of columns
- Column names
- Data types

This is particularly useful when you receive a new dataset.

18. Summary of Data

Use:

```
summary(employees)
```

For numeric columns, R provides values such as:

- Minimum
- 1st Quartile
- Median
- Mean
- 3rd Quartile
- Maximum

This gives us a quick overview of the dataset.

19. Filtering Data

Filtering means selecting records that satisfy a condition.

Suppose we want employees whose salary is greater than ₹40,000.

First:

```
employees$Salary > 40000
```

R returns:

```
FALSE TRUE FALSE TRUE
```

Now use this condition to filter the Data Frame:

```
employees[employees$Salary > 40000, ]
```

Result:

Name	Age	Department	Salary
Rahul	28	IT	45000
Neha	30	IT	52000

20. Filtering by Department

Find IT employees:

```
employees[employees$Department == "IT", ]
```

Find Sales employees:

```
employees[employees$Department == "Sales", ]
```

Find HR employees:

```
employees[employees$Department == "HR", ]
```

21. Filtering Using Multiple Conditions

We can use:

```
& → AND  
| → OR
```

AND Example

Find IT employees earning more than ₹40,000:

```
employees[  
  employees$Department == "IT" &  
  employees$Salary > 40000,  
]
```

Both conditions must be TRUE.

OR Example

Find employees from IT or Sales:

```
employees[
  employees$Department == "IT" |
  employees$Department == "Sales",
]
```

22. Adding a New Column

Suppose we want to calculate annual salary.

```
employees$Annual_Salary <- employees$Salary * 12
```

Now:

```
employees
```

The Data Frame contains a new column:

Name	Salary	Annual_Salary
Amit	30000	360000
Rahul	45000	540000
Priya	38000	456000
Neha	52000	624000

This is called a **derived column** because its values are calculated from another column.

23. Adding a Bonus Column

Suppose every employee receives a 10% bonus.

```
employees$Bonus <- employees$Salary * 0.10
```

Calculate total salary including bonus:

```
employees$Total_Salary <- employees$Salary + employees$Bonus
```

This is a simple example of creating calculated fields in R.

24. Modifying Values

Suppose Amit's salary changes to ₹35,000.

```
employees$Salary[1] <- 35000
```

Check:

```
employees
```

Amit's salary is now 35000.

25. Renaming a Column

Suppose we want to rename `Name` to `Employee_Name`.

```
names(employees)[1] <- "Employee_Name"
```

Check:

```
names(employees)
```

26. Removing a Column

Suppose we don't need the `Bonus` column.

```
employees$Bonus <- NULL
```

The column will be removed.

27. Sorting Data

Suppose we want to sort employees according to salary.

Lowest to highest

```
employees[order(employees$Salary), ]
```

Highest to lowest

```
employees[  
  order(employees$Salary, decreasing = TRUE),  
]
```

This is useful for finding:

- Highest salary
 - Lowest salary
 - Top sales
 - Bottom sales
 - Top-performing employees
-

28. Finding the Highest Value

```
max(employees$Salary)
```

This gives the highest salary value.

To find the complete employee record:

```
employees[which.max(employees$Salary), ]
```

29. Finding the Lowest Value

```
min(employees$Salary)
```

To find the complete employee record:

```
employees[which.min(employees$Salary), ]
```

30. Data Frame vs Vector

Vector

```
salary <- c(30000, 45000, 38000, 52000)
```

A vector contains one series of values.

Data Frame

```
employees <- data.frame(  
  Name = c("Amit", "Rahul", "Priya", "Neha"),  
  Age = c(25, 28, 26, 30),  
  Salary = c(30000, 45000, 38000, 52000)  
)
```

A Data Frame combines multiple variables into one table.

Simple way to remember

Vector = One column

Data Frame = Complete table

31. What is a Tibble?

A **Tibble** is a modern version of a Data Frame used extensively in the **tidyverse** ecosystem.

Example:

```
library(tibble)

employees <- tibble(
  Name = c("Amit", "Rahul", "Priya"),
  Age = c(25, 28, 26),
  Salary = c(30000, 45000, 38000)
)
```

Display:

```
employees
```

32. Data Frame vs Tibble

Data Frame	Tibble
Base R structure	Tidyverse structure
<code>data.frame()</code>	<code>tibble()</code>
Traditional output	Cleaner console output

Base R	Common in modern data analysis
Can contain tabular data	Can contain tabular data

For this course, students should understand both.

Later, when we learn **dplyr** and **ggplot2**, we will work extensively with Tibbles.

33. Complete Practical Example

Let's create a realistic employee dataset.

```
employees <- data.frame(
  Employee_ID = c("E001", "E002", "E003", "E004", "E005"),
  Name = c("Amit", "Rahul", "Priya", "Neha", "Karan"),
  Age = c(25, 28, 26, 30, 27),
  Department = c("Sales", "IT", "HR", "IT", "Sales"),
  Salary = c(30000, 45000, 38000, 52000, 35000)
)
```

View dataset

```
employees
```

Number of employees

```
nrow(employees)
```

Average salary

```
mean(employees$Salary)
```

Highest salary

```
max(employees$Salary)
```

Employees earning above ₹40,000

```
employees[employees$Salary > 40000, ]
```

IT employees

```
employees[employees$Department == "IT", ]
```

Add annual salary

```
employees$Annual_Salary <- employees$Salary * 12
```

Sort by highest salary

```
employees[  
  order(employees$Salary, decreasing = TRUE),  
]
```



DAY 3 PRACTICE QUESTIONS

Practice 1 — Student Data Frame

Create a Data Frame named `students` containing:

- Student_ID
- Name
- Age
- Course
- Marks

Create data for **5 students**.

Perform:

1. Display the Data Frame.
2. Find the number of rows.
3. Find the number of columns.
4. Display column names.
5. Display the first 3 records.
6. Display the last 2 records.

7. Check the structure.
 8. Generate a summary.
-

Practice 2 — Indexing

Using the `students` Data Frame:

1. Display the first student.
 2. Display the third student.
 3. Display only the `Name` column.
 4. Display only the `Marks` column.
 5. Display the first three students.
 6. Display students 2 and 4.
-

Practice 3 — Filtering

Using `students`, find:

1. Students scoring above 70.
 2. Students scoring below 50.
 3. Students scoring exactly 90.
 4. Students aged above 20.
 5. Students scoring above 70 **AND** aged above 20.
-



Practice 4 — Employee Analysis

Create:

```
employees <- data.frame(  
  Name = c("Amit", "Rahul", "Priya", "Neha", "Karan"),  
  Age = c(25, 28, 26, 30, 27),  
  Department = c("Sales", "IT", "HR", "IT", "Sales"),  
  Salary = c(30000, 45000, 38000, 52000, 35000)  
)
```

Perform:

1. Find average salary.
2. Find highest salary.
3. Find lowest salary.
4. Display IT employees.

5. Display employees earning above ₹40,000.
 6. Sort employees by salary from highest to lowest.
 7. Add `Annual_Salary`.
 8. Add a 10% `Bonus`.
 9. Find the employee with the highest salary.
 10. Find the employee with the lowest salary.
-

★ DAY 3 PRACTICAL ASSIGNMENT

Sales Data Analysis

Create the following Data Frame:

```
sales <- data.frame(  
  Order_ID = c("0001", "0002", "0003", "0004", "0005", "0006"),  
  Product = c(  
    "Laptop", "Mouse", "Keyboard",  
    "Laptop", "Monitor", "Mouse"  
  ),  
  Category = c(  
    "Electronics", "Accessories", "Accessories",  
    "Electronics", "Electronics", "Accessories"  
  ),  
  Quantity = c(2, 5, 3, 1, 2, 4),  
  Sales = c(120000, 5000, 7500, 60000, 40000, 4000)  
)
```

Questions

- Q1.** Display the complete dataset.
- Q2.** Find the number of orders.
- Q3.** Calculate total sales.
- Q4.** Calculate average sales per order.
- Q5.** Find the highest sales value.
- Q6.** Find the lowest sales value.
- Q7.** Display orders where sales are greater than ₹20,000.
- Q8.** Display only Electronics orders.
- Q9.** Sort the dataset by Sales from highest to lowest.

Q10. Create a new column:

```
sales$Revenue_Per_Unit <- sales$Sales / sales$Quantity
```

DAY 3 QUICK REVISION

Task	R Code
Create Data Frame	<code>data.frame()</code>
Number of rows	<code>nrow()</code>
Number of columns	<code>ncol()</code>
Dimensions	<code>dim()</code>
Column names	<code>names()</code>
First records	<code>head()</code>
Last records	<code>tail()</code>
Structure	<code>str()</code>
Summary	<code>summary()</code>
Select column	<code>data\$column</code>

Select row	<code>data[1,]</code>
Select cell	<code>data[1, 2]</code>
Filter	<code>data[data\$Salary > 40000,]</code>
Add column	<code>data\$new <- ...</code>
Sort	<code>order()</code>
Highest row	<code>which.max()</code>
Lowest row	<code>which.min()</code>
Create Tibble	<code>tibble()</code>

DAY 3 KEY CONCEPT

The most important syntax to remember is:

```
data[row, column]
```

For example:

```
employees[2, 4]
```

means:

Row 2, Column 4

And:

```
employees[employees$Salary > 40000, ]
```

means:

Give me all rows where Salary is greater than ₹40,000.

This is the foundation for working with real-world datasets in R.